Robo-Editor

Installation & Operations Manual

A local, multi-agent editing studio for manuscripts — Developmental, Line, Copy and Proofread passes, a Co-Author generator, and a KDP Graphics Studio — powered by Ollama running entirely on your own machine.

Build 2026-10-28-r12345 • October 28, 2026
Windows and Linux

Disclaimer and Hold Harmless

This software is provided “as is”, without any warranty of any kind, express or implied, including but not limited to fitness for a particular purpose.

The author and developers accept no responsibility or liability for any problem, loss, or damage arising from this software or its installation — including but not limited to lost or altered files or manuscripts, system issues, or the behaviour of third-party software it works alongside (including Python, Ollama, and Stable Diffusion).

By installing or using this software, you agree to hold the author and developers harmless from any claims, damages, or expenses arising out of that installation or use. You install and use it entirely at your own risk, and you are responsible for keeping your own backups of your manuscripts and files.

> This software is a tool. Always work on a copy of your manuscript, never your only original. The same disclaimer ships with the software itself as DISCLAIMER.txt.

Contents

1 What the Robo-Editor is

2 System requirements

3 Installation — Windows

4 Installation — Linux

5 Optional plugin — Stable Diffusion artwork

6 First run and verification checklist

7 Operating the editor

8 Running an edit, step by step

9 The Co-Author agent

10 Bible file and style directives

11 Graphics Studio — book spec and covers

12 Graphics Studio — interior graphics and art prompts

13 Graphics Studio — Text to Graphic

14 Safety: how the editor protects your manuscript

15 Files and settings reference

16 Troubleshooting

17 Updating to a new build, and uninstalling

Appendix Command cheat sheet

1 What the Robo-Editor is

The Robo-Editor edits a Word manuscript (.docx) in chunks, using a language model that runs locally through Ollama. Nothing is sent to the cloud. Four agents divide the work:

Agent	Job
Keeper	Maintains the rolling continuity ledger and loads your story bible, so later chunks are edited with knowledge of names, facts, and canon established earlier.
Scribe	Performs the actual editing pass for the stages you select.
WatchTower	Audits every edited chunk before it is accepted: word-count drift limits, names, numbers, key terms, headings, quote style, banned vocabulary, and an AI audit. A chunk that fails is sent back for correction (up to two attempts); if it still fails, the original text is used instead.
Co-Author	Generates new sections from a prompt or from a topic document, audited by the WatchTower in the same way.

The Graphics Studio (opened from the main window) produces KDP-ready book graphics: print cover wraps (PNG, and PDF for upload), ebook covers, print-ready interior images, cover-art prompts, and graphics generated from your own manuscript text.

> **The golden rule:** the pipeline is built so its worst case is your unedited text — never a damaged manuscript. Section 14 explains the protections in detail.

2 System requirements

Item	Requirement
Operating system	Windows 10/11, or a current Linux distribution with a desktop (the program opens a window; a headless server will not do).
Python	Python 3.9 or newer. Python 3.11 or 3.12 is recommended.
Ollama	A working local Ollama installation serving at least one downloaded model.
Memory / disk	8 GB RAM minimum, 16 GB recommended. Each 8B model occupies roughly 5–9 GB of disk.
Graphics card	Optional but strongly recommended for speed. An 8B model runs well on a 12 GB card; CPU-only works, slowly.
Manuscript format	Input and output are .docx (Word) files. For books with images, edit a no-graphics copy — see Section 14.
Optional	AUTOMATIC1111 Stable Diffusion (Section 5) for photographic cover art. Not required: the Studio has a built-in art engine.

3 Installation — Windows

Step 1 — Install Python

- Download Python 3.12 (or 3.11) from python.org and run the installer.

- On the first installer screen, tick **“Add Python to PATH”** before clicking Install. This is the step everyone misses.
- Verify in a new Command Prompt or PowerShell window:

```
python --version
```

You should see Python 3.12.x (or similar). The python.org installer includes Tk, which the window needs — nothing extra to install.

Step 2 — Install Ollama and a model

- Download and run the Ollama installer from ollama.com. Ollama then runs in the background and answers on port 11434.
- Verify, and download a model (about 5 GB for an 8B model):

```
ollama --version
ollama pull qwen3:8b
ollama list
```

Recommended models for this editor: **qwen3:8b** (best instruction-following in its class; tick the editor's No-think option when using it) or **llama3.1:8b** (a zero-fuss alternative). Any model shown by *ollama list* will appear in the editor's model menu.

Step 3 — Extract the Robo-Editor

- Unzip robo-editor-rounds12345.zip into a folder of your own, for example Documents\RoboEditor.
- Keep the folder structure exactly as unzipped: app.py, engine.py, orchestrator.py and the graphics modules at the top; agents\ and tests\ as subfolders. Do not flatten the folders.

Step 4 — Install the Python prerequisites

- Open Command Prompt in that folder (type cmd in the Explorer address bar) and run:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

The requirements are pinned: customtkinter 6.0.0 (the window), python-docx 1.1.0 (Word files), tiktoken 0.14.0 (accurate chunk sizing), Pillow 10.2.0 (graphics).

Step 5 — Run it

```
python app.py
```

The main window opens and, a moment later, the model menu fills with your Ollama models. Section 6 is a short verification checklist to run before your first real book.

4 Installation — Linux

Step 1 — Install Python and Tk

- Debian / Ubuntu:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-tk
```

- **python3-tk is not optional.** Without it the program cannot open a window and fails with a Tkinter import error. This is the most common Linux install problem.
- Fedora: sudo dnf install -y python3 python3-tkinter. Then verify:

```
python3 --version
```

Step 2 — Install Ollama and a model

```
curl -fsSL https://ollama.com/install.sh | sh
systemctl status ollama   (it installs as a service)
ollama pull qwen3:8b
ollama list
```

If the service is not running: sudo systemctl enable --now ollama. Ollama answers on port 11434, the same as on Windows.

Step 3 — Extract, create a virtual environment, install prerequisites

```
unzip robo-editor-rounds12345.zip -d ~/RoboEditor
cd ~/RoboEditor
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Step 4 — Run it

```
python3 app.py
```

Run it from a desktop session — the program needs a display. (On a headless machine you can test with xvfb-run, but for real work use your desktop.)

5 Optional plugin — Stable Diffusion artwork

The Graphics Studio's Cover Builder can paint cover art three ways: a local Stable Diffusion server, its own built-in art engine, or an art file you supply. Stable Diffusion is the photographic option and is entirely optional.

- **Windows:** install AUTOMATIC1111 (github.com/AUTOMATIC1111/stable-diffusion-webui), place a checkpoint model in its models\Stable-diffusion folder, and launch it with the API enabled:

```
webui-user.bat — set COMMANDLINE_ARGS=--api
```

- **Linux:** clone the same repository, place a checkpoint likewise, and run:

```
./webui.sh --api
```

- Confirm it is listening on port 7860 — the Studio checks http://127.0.0.1:7860 automatically.

How the Studio uses it. With Artwork set to “Auto (AI if running, else built-in)”, the Studio uses your art file if one is set; otherwise Stable Diffusion if the server answers; otherwise the built-in engine. The status line always names which one actually painted, and says so honestly if the server was found but the generation failed.

Resolution note. Stable Diffusion art is generated capped at about 1344 px on the long edge (sized for a 12 GB card). A print wrap is 3873 px wide, so SD art is upscaled roughly 2.9× (about 104 effective DPI) and the status line warns you when that happens. For a final print cover, supply full-resolution art

through the Art file option, or use SD for the ebook cover, where it is close to native size.

6 First run and verification checklist

- 1. Start Ollama (Windows: it runs after install; Linux: systemctl status ollama).
- 2. Launch the editor. Within a few seconds the model menu lists your models. If it reads “Connection Error”, Ollama is not running — fix that first (Section 16).
- 3. Pick your model in the menu. Your choice is remembered between sessions.
- 4. Prove the install with the built-in test suite (no Ollama or manuscript needed). Windows: python -m unittest discover -s tests • Linux: python3 -m unittest discover -s tests. The full suite should report OK (a handful of app-window tests skip if the GUI toolkit is not loaded in that terminal — that is expected).
- 5. Do a first edit on a **copy** of a short document — a chapter, not a whole book — and read the output before trusting the pipeline with a full manuscript.

Settings are stored in robo_config.json in the program folder and are written safely: the file is replaced atomically and merged, so the editor and the Graphics Studio can never wipe each other's settings.

7 Operating the editor

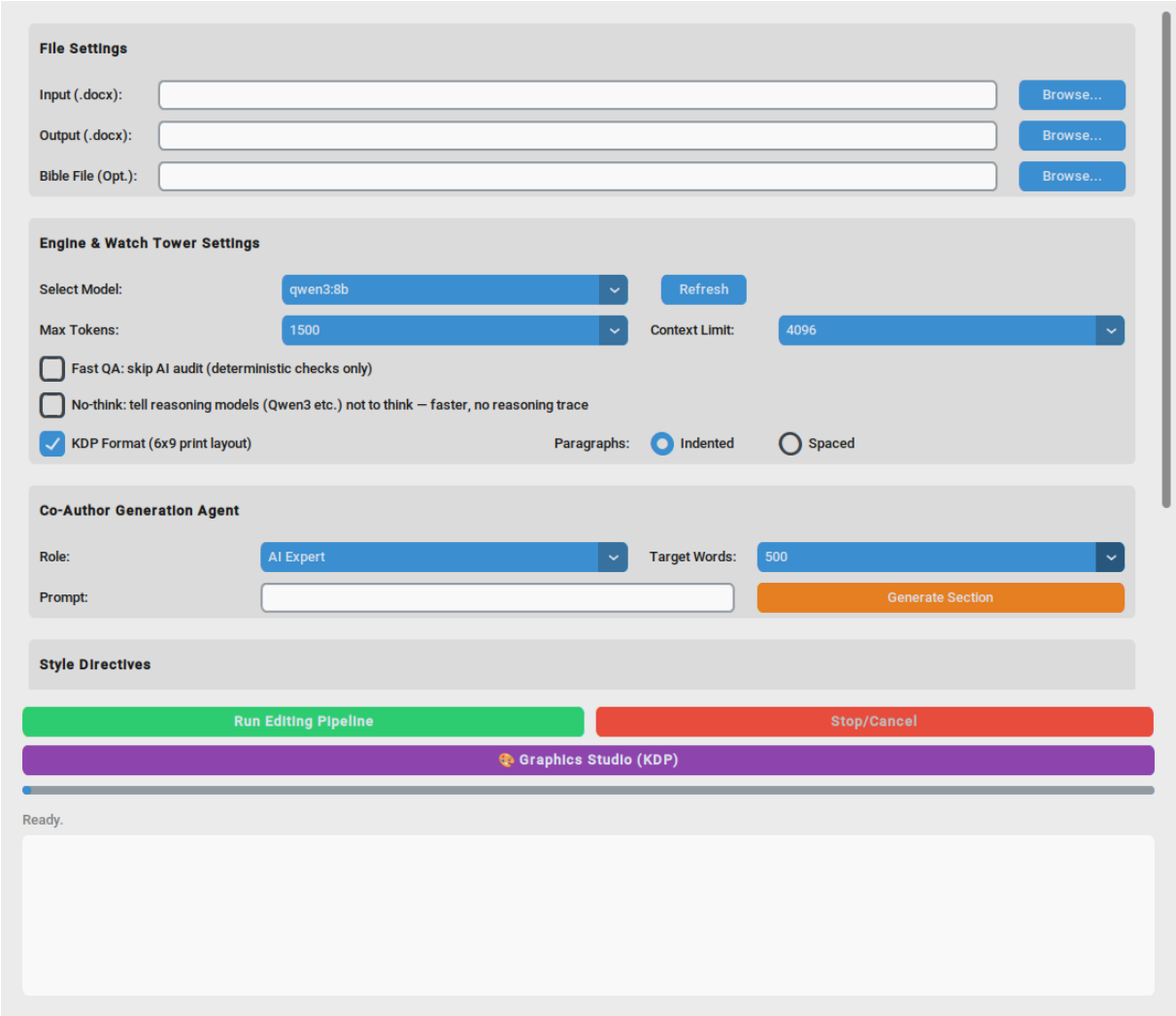


Figure 1 — The main window: File Settings, Engine & Watch Tower Settings, the Co-Author panel, and the run controls.

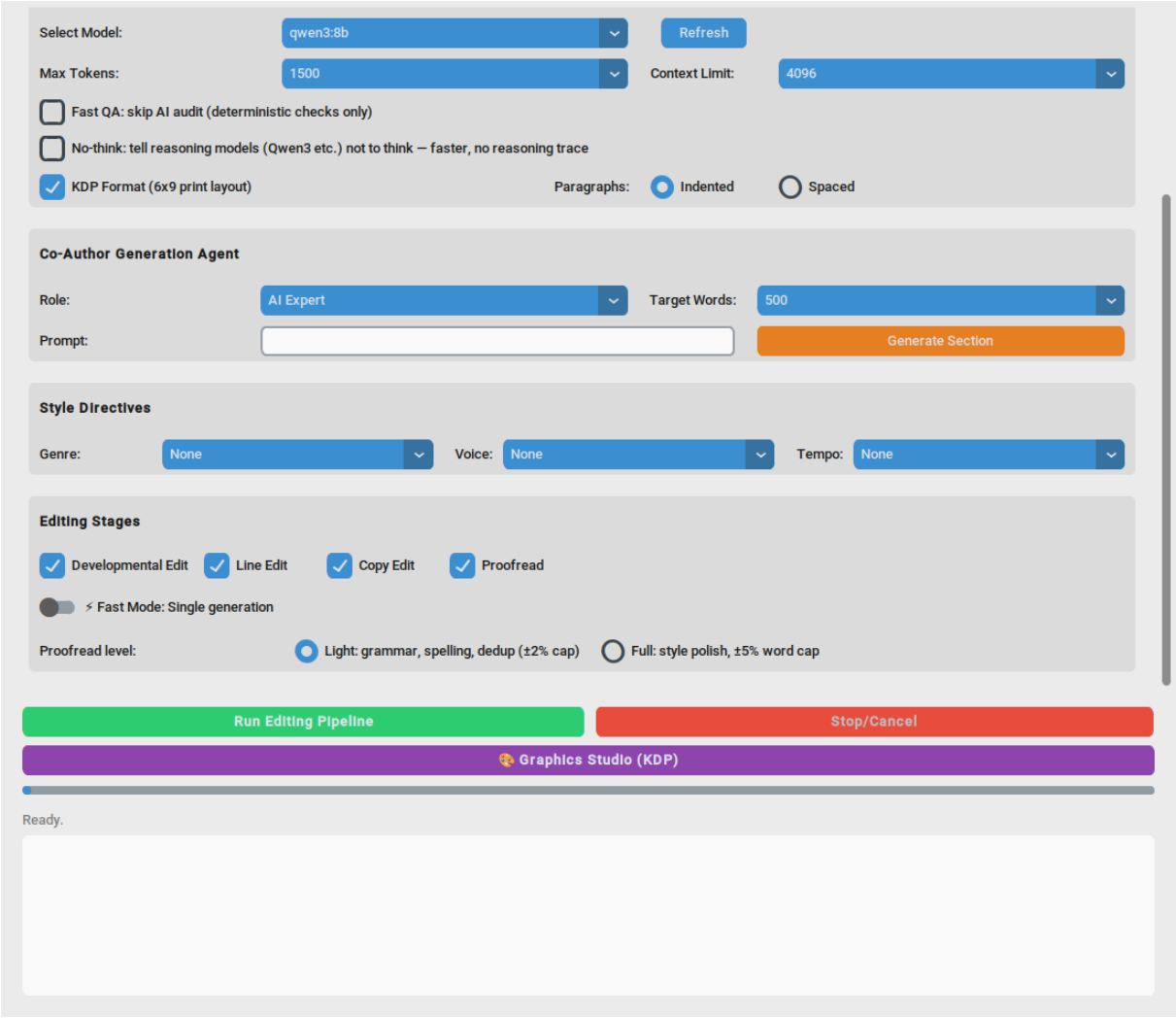


Figure 2 — Style Directives (Genre, Voice, Tempo), the Editing Stages checkboxes, Fast Mode, and the Proofread level.

Control	What it does
Input (.docx)	The manuscript to edit. Browsing fills the Output automatically with a safe "_edited" name.
Output (.docx)	Where the edited book is written. It may not be the input file — the editor refuses, so a run can never overwrite its own source. Check hand-typed paths: you are not prompted if the file already exists.
Bible File (Opt.)	A .docx or .txt reference (story bible, style sheet). Its canon is given to the Scribe and used by the Keeper.
Select Model	Any model from your Ollama. Refresh re-reads the list.
Max Tokens	Chunk size: 500–2000 (default 1500). Smaller chunks are gentler and slower; larger chunks give the model more context per pass.
Context Limit	2048 / 4096 / 8192 — how much surrounding text accompanies each chunk.
Fast QA	Skips the AI audit; deterministic WatchTower checks (drift, names, numbers, terms) still run. Faster, less thorough.

Control	What it does
No-think	Tells reasoning models (Qwen3 and similar) not to emit a reasoning trace. Faster, and keeps reasoning text out of your book. Recommended for Qwen3.
KDP Format	Lays the output out as a 6×9 print interior. Paragraphs: Indented (fiction style) or Spaced (non-fiction style).
Editing Stages	Developmental Edit, Line Edit, Copy Edit, Proofread — any combination. Stages run in that order over every chunk.
Fast Mode	Single generation per chunk instead of the full audited correction loop. Much faster; use for drafts, not finals.
Proofread level	Light — grammar, spelling, dedup, word count held within ±2%. Full — adds style polish, ±5% cap.

8 Running an edit, step by step

- 1. Work on a copy. Browse to the copy as Input; confirm the Output name.
- 2. Choose the model, stages, and proofread level. For a first full pass, Proofread alone at Light is the gentle option; Combined behaviour comes from ticking several stages.
- 3. Click **Run Editing Pipeline**. The log narrates each chunk: stage, WatchTower verdicts, corrections, and fallbacks.
- 4. Let it finish. “Saving final document…” is followed by a post-save audit that re-reads the finished book and reports any problems it finds (missing sections, drift, invented names) in the log.

Stopping, and closing the window

Stop/Cancel asks the current operation to finish safely and writes a checkpoint. Closing the window mid-run now asks for confirmation first; if you confirm, the editor requests a cancel and waits for the worker to stop before closing — a run is never killed half-way through a save.

Checkpoints and resuming

Every five chunks, and whenever a run is interrupted, the editor writes two sidecar files next to your output: a .progress.json checkpoint and a .partial.docx you can open and inspect. Re-run with the same input and the same settings and the run resumes where it stopped. If the input file changed, or the model, stages, chunk size, or style changed, the checkpoint is refused and the log tells you it is starting fresh — resuming across changed settings would silently mix two different edits. On a successful finish the sidecars are removed.

> A chunk whose model call fails outright (Ollama stopped, connection lost) no longer kills the run: that chunk falls back to its original text, the log says so, and processing continues.

9 The Co-Author agent

- Set a **Role** (AI Expert, Engineer, Novelist, Researcher) and a **Target Words** count.
- Type a **Prompt**, or leave it blank and point Input at a .docx whose text is the topic.
- Click **Generate Section**. Output streams into the log and is saved to your Output file; partial saves are written every five blocks.

Co-Author output is audited by the WatchTower like edited text, and a rolling ledger keeps established facts straight across blocks, so a long generated book does not contradict its own early chapters. Co-Author is a drafting tool: read what it writes before it goes anywhere near print.

10 Bible file and style directives

The **bible file** is your canon lock: names and spellings, facts that must not change, terminology. The Keeper caches it and the Scribe is instructed not to contradict it. Keep it factual and current — the editor treats it as ground truth.

Genre, Voice, and Tempo are light-touch style directives layered over the stage instructions. “None” means: edit without steering the author's voice. When in doubt, leave them at None — the WatchTower protects the text either way.

11 Graphics Studio — book spec and covers

Open it with the purple **Graphics Studio (KDP)** button. One window only — clicking again raises it.

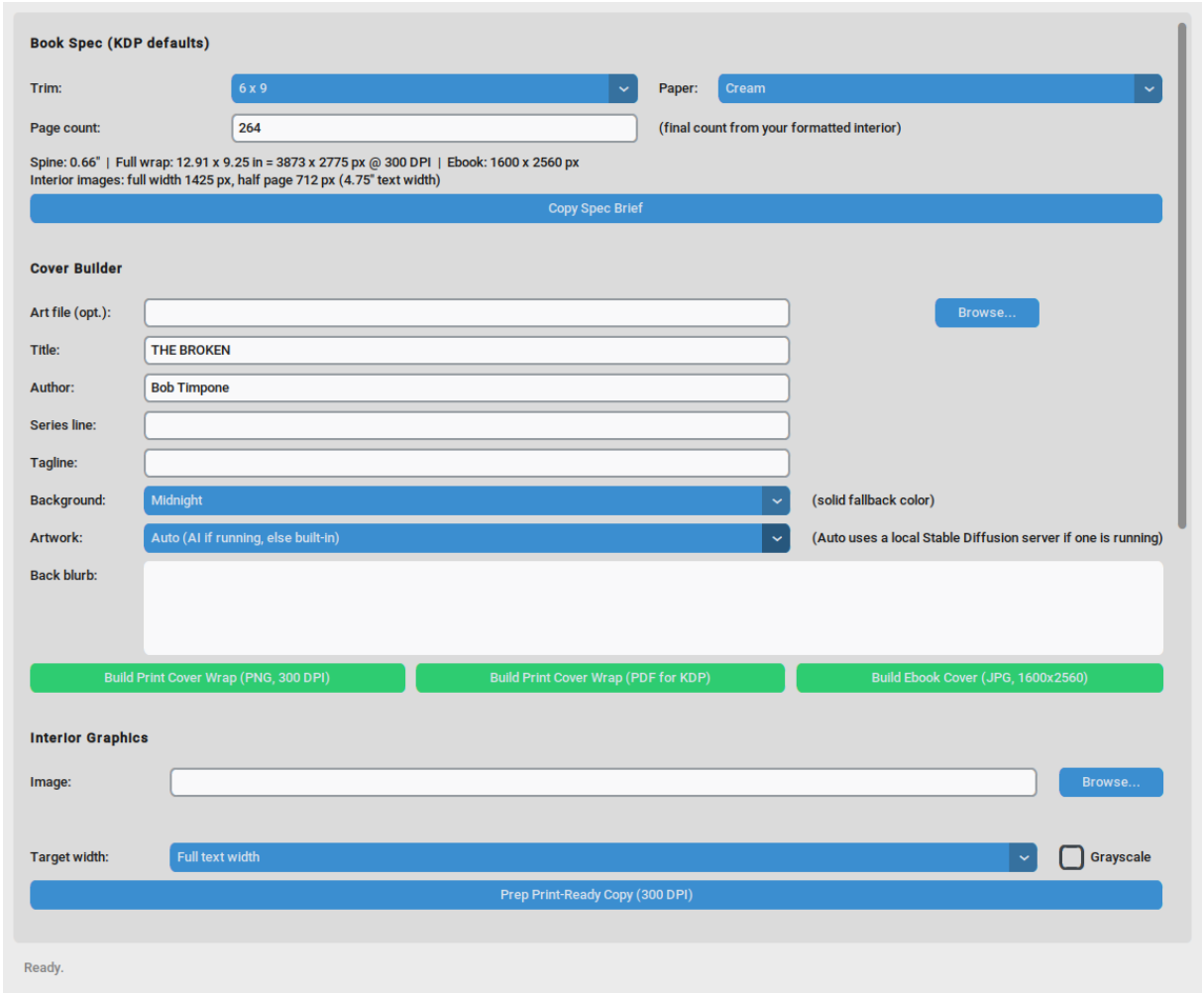


Figure 3 — The Graphics Studio: Book Spec (here, THE BROKEN at 264 pages: 0.66″ spine, 12.91 × 9.25 in wrap, 3873 × 2775 px), the Cover Builder with its Artwork selector, and the three build buttons — PNG wrap, PDF wrap for KDP, and ebook JPG.

Book Spec

Set Trim, Paper, and the final Page count from your formatted interior. The spec line computes the spine width (pages × the paper's per-page factor), the full wrap size with 0.125″ bleed, the ebook size, and your interior image widths. **Copy Spec Brief** puts the whole spec on the clipboard for a cover designer. Page counts outside KDP's 24–828 range, and other invalid specs, are refused with a clear message.

Interior widths respect KDP's gutter: the required inside margin grows with page count (0.375″ up to 150 pages, rising to 0.875″ at 701–828), and “full text width” is computed from whichever is larger — your margin profile or the requirement — with a warning when the requirement is the one that bites.

Cover Builder

- **Title, Author, Series line, Tagline, Back blurb** — the cover's text. Text is laid out inside KDP's safe zone, measured from the trim edge.
- **Artwork: Auto** — your art file if set, else a running Stable Diffusion server, else the built-in engine. Or choose Built-in, Art file, or Solid background directly.
- **Built-in art engine** — paints a layered illustration (sky, celestial body, terrain, and a hero silhouette picked from your prompt words: drop pod, starship, sea, tower, skyline, forest, or a lone figure). It is deterministic: the same title, subject, genre, and mood always paint the same picture, so series covers can share a look.
- **Buttons** — Build Print Cover Wrap (PNG, 300 DPI) for proofing; **Build Print Cover Wrap (PDF for KDP)** for the file you actually upload — KDP paperback covers must be a single PDF at the exact wrap size; Build Ebook Cover (JPG, 1600 × 2560).
- **Spine text** is fitted to the real spine: the font shrinks to fit the usable width, and on a thin spine (short books) it is omitted with a warning rather than spilling onto the covers. Long blurbs are trimmed above the barcode box with a warning — read the status line after every build; it reports warnings, it does not hide them.
- A named art file that does not exist stops the build with an error. It never silently becomes a plain-colour cover.

12 Graphics Studio — interior graphics and art prompts

Interior Graphics

Browse to an image and choose a target width (full text width, half page, and other spec widths). **Prep Print-Ready Copy** writes a copy at the exact pixel width for your trim at 300 DPI, optionally in grayscale. Note it cannot invent resolution: a small source enlarged to print width will be stamped 300 DPI but will not gain detail — start from the largest source you have.

Cover Art Prompt

Fill Subject, Setting, Mood, Style, and Genre, then **Build Prompt**. The composer produces a prompt and a negative prompt sized and phrased for an external image generator (including the local Stable Diffusion path), with the composition warnings a book cover needs (room for the title, no text in the art). **Copy Prompt** puts it on the clipboard.

13 Graphics Studio — Text to Graphic

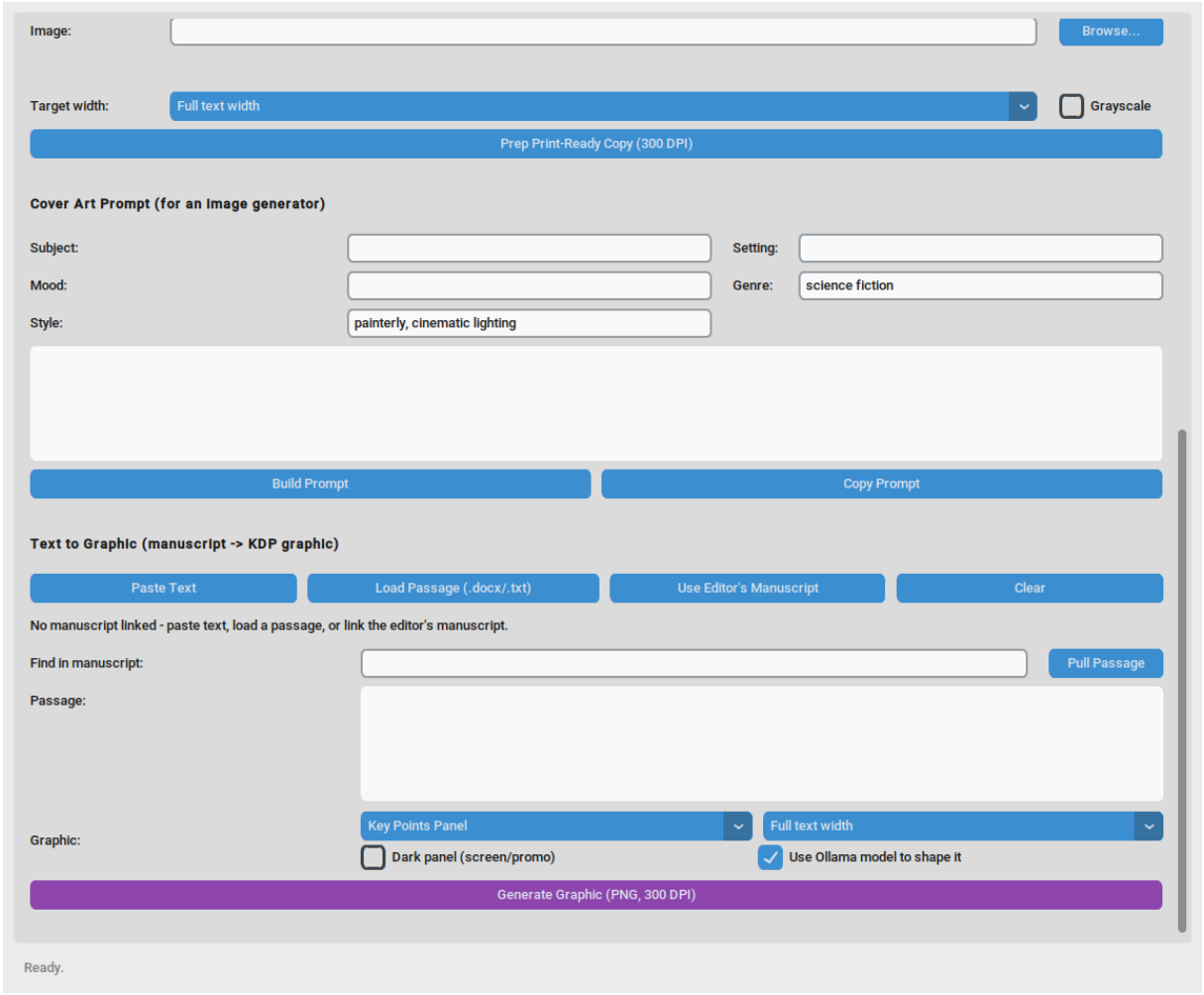


Figure 4 — Text to Graphic: paste text, load a passage file, or link the editor's manuscript and pull a passage by search.

Text to Graphic turns a passage of your manuscript into a print-ready panel (PNG, 300 DPI) at full text width, half page, or front-cover width.

- **Get text in:** Paste Text (from the clipboard), Load Passage (.docx / .txt / .md), or Use Editor's Manuscript to link the main window's Input — then type a name, place, or phrase in “Find in manuscript” and Pull Passage returns the paragraph it lives in, plus one either side. With a manuscript linked and nothing pulled, Generate works from the opening.
- **Five kinds:** Key Points Panel, Pull-Quote Card, Chapter Callout, Scene Card, Timeline.
- **Two brains:** with “Use Ollama model to shape it” ticked, the main window's model structures the passage. Its output is fenced: a quote must be a verbatim sentence from your passage or it is replaced with one, and bullets must genuinely come from the passage or they are dropped. If Ollama is off, unreachable, or its answer cannot be trusted, a deterministic built-in extractor does the job offline — and the status line tells you which brain ran.
- **Dark panel** switches to the screen/promo style; the default light panel is grayscale-safe for print interiors.

KEY POINTS PANEL

The nanites had repaired Jex eleven times

• The nanites had repaired Jex eleven times, and each repair left him a little less certain which memories were his.
• Below the ridge, the fragment pods came down through the ash like slow rain.
• Sarge counted forty postings on the board and crossed out three names without speaking.
• The Alliance would stamp the form DENIED in the morning.
• Tonight there was only grey soup, mud, and the long work of keeping the Draven alive.

“The nanites had repaired Jex eleven times, and each repair left him a little less certain which memories were his.”

Generated from manuscript text

Figure 5 — A Key Points Panel produced from a passage with the built-in extractor. Every bullet and the quote are the passage's own sentences.



Figure 6 — An ebook cover built with the built-in art engine (seed: a fragment pod over a ruined station). Deterministic: rebuilding with the same words reproduces this picture.

14 Safety: how the editor protects your manuscript

Protection	What it means in practice
Do-no-harm fallback	A chunk that fails the WatchTower — or whose model call fails — is saved as its original, unedited text. The worst case is an unedited chunk, never a lost one.
Gate checks	Drift caps per stage; names, numbers, and key terms may not be invented, renamed, or dropped (counted, not just spotted — one of two identical headings going missing is caught); quote style is preserved; banned vocabulary is judged against your original, so your own words are never held against you.
Save-time backstop	A final sanitizer strips model scaffolding. If it would remove a real share of any chunk, that chunk is saved unsanitized instead, with a loud warning in the log.

Protection	What it means in practice
Atomic saves	Manuscripts, checkpoints, and the settings file are written to a temporary file and then replaced in one step. A crash or power cut mid-save leaves the previous good file, not a truncated one.
Input protection	Output may not equal input; settings changes mid-run cannot affect the run in progress (settings are snapshotted when you click Run).
Post-save audit	The finished book is re-read and graded against the original; problems are reported in the log before you ever open the file.

Working rules that still apply: keep backups; edit copies, not originals; and for books with images, edit a no-graphics copy — the editor rebuilds the document's text and does not carry images, headers/footers, footnotes, or tracked changes across. You will receive a warning with the image count when it loads a manuscript that contains pictures.

15 Files and settings reference

File	Purpose
app.py	The main window. Run this.
engine.py	Document loading/saving, chunking, Ollama calls, atomic file writes.
orchestrator.py	Runs the agents per chunk; owns the co-author ledger.
agents/	keeper_agent.py, scribe_agent.py, watchtower_agent.py, coauthor_agent.py, vocab.py (shared stage names and gate vocabulary).
graphics_page.py	The Graphics Studio window.
graphics_kdp.py	KDP spec maths and the cover / interior-image builders (PNG, JPG, PDF).
graphics_art.py	The built-in art engine and the Stable Diffusion client.
graphics_text.py	Text to Graphic: passage loading, passage pull, analysis, rendering.
requirements.txt	Pinned Python prerequisites.
robo_config.json	Your settings (created on first close). Editor and Studio share it safely; if it is ever unreadable, saves preserve it untouched rather than wiping it.
DISCLAIMER.txt	The disclaimer from the front of this manual.
.progress.json	Resume checkpoint (sidecar, removed on success).
.partial.docx	The work-in-progress book at the last checkpoint; safe to open and inspect.
tests/	The built-in test suite (Section 6).

16 Troubleshooting

Symptom	Likely cause and fix
Model menu says “Connection Error”	Ollama is not running. Windows: start Ollama from the Start menu. Linux: sudo systemctl start ollama. Then click Refresh. Do not start a run against the “Connection Error” entry — it is not a model.
Model menu is empty	Ollama is running but has no models. Run: ollama pull qwen3:8b — then Refresh.
Program will not start on Linux: No module named tkinter	Install the Tk package: sudo apt install python3-tk (Fedora: python3-tkinter).
“No module named customtkinter” (either OS)	The prerequisites are not installed in the environment you launched from. Activate the virtual environment first, then pip install -r requirements.txt, then run.
A run starts fresh instead of resuming	By design, if the input file, model, stages, chunk size, or style changed since the checkpoint — or the checkpoint file was damaged. The log states the reason.
A full-book run is slow	Expected on CPU; a graphics card helps greatly. Each WatchTower rejection costs extra model calls (a correction plus a re-audit), so strict stages on a first pass are the slowest combination. Fast Mode and Fast QA exist for drafts.
Reasoning text appears in the output (Qwen3)	Tick No-think in the main window for reasoning models.
Studio status: “no SD server found”	AUTOMATIC1111 is not running with --api, or not on port 7860. The built-in engine painted instead — the status line says which.
Studio status names an SD failure	The server answered but generation failed (often out of video memory). Restart it, lower other GPU load, or use Built-in / Art file.
Cover build stops: art file not found	The Art file path is stale (moved or renamed file). Re-browse to it, or clear it and use Auto.
No spine text on the print wrap	The spine is too thin for legible text at your page count — normal for short books; KDP does not allow spine text under 79 pages in any case.
PDF wrap: page size looks wrong in a viewer	The PDF is written at the exact wrap size in inches (for 6×9, 264 pages, cream: 12.91 × 9.25 in). Check the viewer's page-size readout, not its zoom.
Settings seem lost after a crash	Settings save when the window closes. A crash before closing loses that session's changes only — the config file itself is never corrupted by a crash, because writes are atomic.

17 Updating to a new build, and uninstalling

- **Update:** extract the new build into a *new* folder, install its requirements into its own virtual environment, and copy your robo_config.json across if you want your settings. Keep the old folder until you have run the new build once — builds are self-contained and do not modify each other.
- **Uninstall:** delete the program folder and, if you no longer want it, Ollama and its models (Windows: uninstall from Settings; Linux: systemctl disable ollama and remove ~/.ollama). Your manuscripts and outputs are yours and are never touched by uninstalling.

This manual describes build 2026-10-28-r12345. The program's log line at the start of every run names the build that produced it — quote it when reporting a problem.

Appendix Command cheat sheet

Task	Windows	Linux
Check Python	python --version	python3 --version
Install Ollama	Installer from ollama.com	curl -fsSL https://ollama.com/install.sh | sh
Ollama running?	ollama list	systemctl status ollama
Get a model	ollama pull qwen3:8b	ollama pull qwen3:8b
Create venv	python -m venv .venv	python3 -m venv .venv
Activate venv	.venv\Scripts\activate	source .venv/bin/activate
Install prerequisites	pip install -r requirements.txt	pip install -r requirements.txt
Run the editor	python app.py	python3 app.py
Run the tests	python -m unittest discover -s tests	python3 -m unittest discover -s tests
Start SD with API	webui-user.bat (COMMANDLINE_ARGS=--api)	./webui.sh --api

— End of manual —